Fyzické operátory

Ján Mazák

FMFI UK Bratislava

Disky / storage media

Dáta čítame a zapisujeme po blokoch (stránkach), predvolená veľkosť bloku v PostgreSQL je 8 kB.

- HDD: náhodný prístup stojí pohyb hlavy a otáčanie platní
- SSD: bez mechaniky, náhodný prístup je rýchly; zápis ide po stránkach, mazanie po väčších blokoch a bunky vydržia len niekoľko tisíc zápisov
- páska: len sekvenčný prístup, na požadované miesto sa treba pretočiť; používa sa najmä na lacné zálohovanie

`https://spectrum.ieee.org/`

`why-the-future-of-data-storage-is-still-magnetic-tape`

Disky / storage media

Ilustratívne hodnoty:

Médium	Náhodný prístup	Podiel na trhu cloudu	CapEx [USD/TB]	TCO za 5 r. [USD/TB]
HDD	8 ms	82 %	18	45
SSD	0,1 ms	12 %	105	135
Páska	60 000 ms	6 %	6	14

SSD je pri nákupe približne 6x drahší, za 5 rokov 3x (spotreba, miesto v racku, vibrácie, výmeny).

Pointa: optimalizujeme pre HDD — minimalizovať náhodné prístupy a čítať sekvenčne po blokoch. PostgreSQL to odráža v cenovom modeli: `random_page_cost` = 4 oproti `seq_page_cost` = 1 (predvolené hodnoty).

Disky / storage media

Triky na urýchlenie:

- uložiť často čítané bloky do cache
- načítať vopred bloky, ktoré pravdepodobne bude treba v blízkej budúcnosti
- zbierať dáta do vyrovnávacej pamäte (buffer) a zapísať naraz väčšie množstvo

Toto funguje aj mimo DBMS, ale pri operáciách veľkého rozsahu sa oplatí špecificky optimalizovať
(napr. AWS Caching Solutions).

Fyzické operátory

Zápis dotazu v relačnej algebre určuje postup výpočtu, ale len rámcovo. Treba zohľadniť aj fyzické uloženie dát. Každý operátor relačnej algebry má minimálne jeden zodpovedajúci fyzický operátor, často však niekoľko — vyjadrujú rôzne spôsoby výpočtu.

Idey týkajúce sa fyzických operátorov sú užitočné všeobecne aj mimo DBMS — väčšina problémov súvisiacich s veľkým množstvom dát sa dá riešiť skombinovaním hashovania, usporadúvania, filtrovania a vhodného výpočtu joinov.

Fyzické operátory

Algoritmy fyzických operátorov sa snažia minimalizovať počet diskových operácií, nie kroky výpočtu v operačnej pamäti.

Mnohé operátory sú implementované ako iterátory: priebežne spracúvajú záznamy zo vstupu a vytvárajú výstup (*pipelining*). Výhoda: netreba ukladať medzivýsledky na disk.

Pipelining a generátory v Pythone

Operátory si môžeme predstaviť ako zreťazené generátory:

Operátor v DBMS	**Generátor v Pythone**
Požiadavka na ďalší riadok	`next(g)`
Odovzdanie jedného riadka	`yield row`
Pokračovanie výpočtu	Ďalší `next(g)` nadviaže za `yield`; lokálny stav ostáva zachovaný.
Zreťazenie operátorov	Generátor číta z iného generátora.

Filter môže vydávať vyhovujúce riadky priebežne.

Triedenie je blokujúce: podobne ako `sorted(g)` musí pred prvým výstupom spracovať celý vstup.

Fyzické operátory — triedenie

Množstvo triedených dát rádovo prevyšuje dostupnú operačnú pamäť. Riešenie: externý merge sort.
Majme B blokov v RAM a N blokov dát na disku. Dve fázy:

1. Prečítame B blokov dát, usporiadame v RAM, zapíšeme na disk. Opakujeme, kým máme dáta. Vznikne $\lceil N/B \rceil$ usporiadaných „behov“ (runs).
2. Pre výstup rezervujeme 1 blok RAM (až sa naplní, zapíšeme ho na disk). Zlúčime $B - 1$ behov tak, že pre každý rezervujeme 1 blok (keď sa načítané dáta minú, prečítame ďalší blok tohto behu z disku). Pri určovaní záznamu, ktorý patrí na výstup, sa stačí pozrieť na prvý záznam každého behu. Opakujeme, kým neostane jediný beh.

Fyzické operátory — triedenie

Zložitosť externého merge sortu: vo všeobecnosti

$$2N \cdot (1 + \lceil \log_{B-1}(\lceil N/B \rceil) \rceil),$$

závisí však od dostupnej pamäti.

Úloha. Koľko pamäti B v závislosti od N treba, aby 2. fázu stačilo vykonať raz a nebolo potrebné ju opakovať? Koľko I/O operácií na blokoch sa vtedy vykoná?

Fyzické operátory — triedenie

Úloha. Relácia s 10^7 záznamami je uložená na disku v blokoch veľkosti 8 kB. V každom bloku je 100 záznamov relácie. K dispozícii nie je žiaden index. Prenos diskového bloku (do RAM aj z RAM) trvá 1 ms. Bloky v RAM a na disku sú rovnako veľké. Na usporiadanie záznamov použijeme merge sort.
(a) Ako dlho bude trvať usporiadanie záznamov tejto relácie, ak máme alokovaných v RAM 200 blokov?
(b) Ako dlho bude trvať usporiadanie záznamov tejto relácie, ak máme alokovaných v RAM 2000 blokov (čiže 10x viac)?

Fyzické operátory — join / antijoin

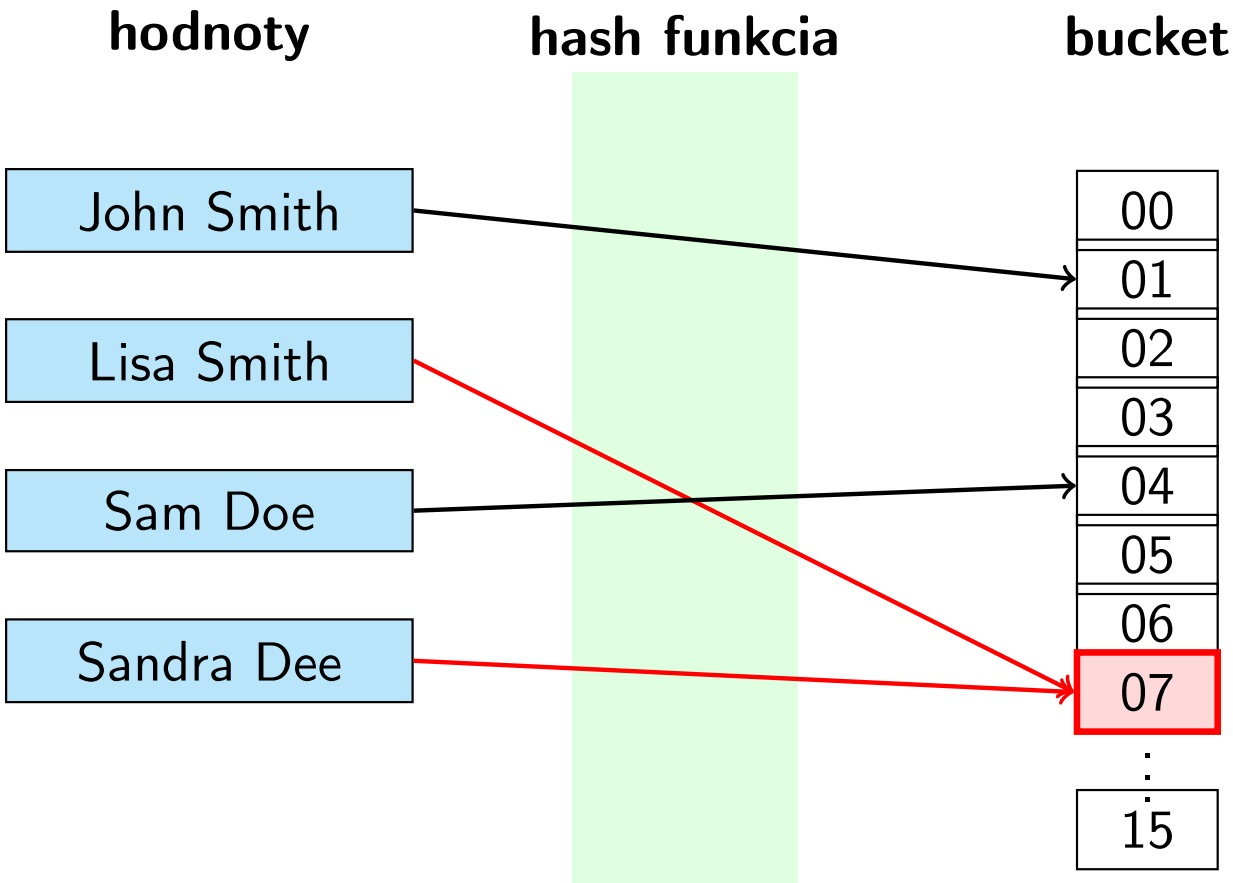
- nested loop join — dva vnorené cykly párujú každý záznam s každým
- merge join — ak joinovacia podmienka s rovnosťou obsahuje atribúty, podľa ktorých obe joinované relácie vieme usporiadať, join sa ráta ľahko podobným spôsobom, ako zlučovacia fáza merge-sortu (výhodné kombinovať s index scanom, ktorý produkuje záznamy už správne usporiadané)
- hash join — pri podmienke s rovnosťou: nutnou podmienkou na join dvoch záznamov z jednotlivých relácií je, že patria do rovnakých bucketov (nie je to však postačujúce), vieme teda join rátať bucket po buckete; hashovacie buckety vytvoríme nanovo špeciálne pre účely joinu

Fyzické operátory — merge join

- Predpoklad: obe relácie sú usporiadané podľa stĺpcov v podmienke, napr. `R.a = S.a` (ak nie, najprv ich utriedime).
- Dva ukazovatele postupujú zaradom: pri rôznych hodnotách sa vpred posunie ten s menšou; pri rovnakej hodnote v oboch reláciách sa spárujú všetky riadky s touto rovnakou hodnotou.
- Po utriedení stačí jedno čítanie oboch relácií; pri duplicitných hodnotách treba dočasne uchovať celú skupinu riadkov s rovnakou hodnotou, aby sme ju spárovali so všetkými riadkami druhej relácie.
- Ak usporiadanie už máme z indexu, usporadúvanie odpadá. (Index podľa atribútu pomáha aj pri `BETWEEN` a `ORDER BY`, ale merge join počítame len pre rovnosť.)

Hashovanie

Rozdelíme záznamy do skupín podľa hodnoty vypočítanej hashovacou funkciou.



Červeným je označená *kolízia*.

Hashovanie

„Dobrá“ hashovacia funkcia:

- rozdeľuje rovnomerne
- využíva celý obor hodnôt
- dá sa počítať veľmi rýchlo

Kolízie nám veľmi neprekážajú, použijeme spájaný zoznam.

(Rozdiel oproti tzv. kryptografickým hashovacím funkciám, kde obtiažnosť algoritmického hľadania kolízií je kľúčová.)

Hash join

Ku každej udalosti chceme doplniť meno používateľa. Ide o všeobecný problém spájania dát, nielen relačných databáz: rovnako spájame napr. dva CSV súbory.

Udalosti R

udalosť	user_id
e_1	17
e_2	42
e_3	17
e_4	99

Používatelia S

user_id	meno
17	Eva
42	Adam
58	Nina

```
SELECT R.udalost, S.meno FROM udalosti R
    JOIN pouzivatelia S ON R.user_id = S.user_id;
```

Predpokladajme, že udalostí (R) je oveľa viac než používateľov (S).

Hash join — hash buckets

Hašovacia funkcia priradí hodnote hash bucket, napr.

$$h(k) = k \bmod 3.$$

hash bucket	používatelia uložení v hašovacej tabuľke
0	(42, Adam)
1	(58, Nina)
2	(17, Eva)

- Rovnaké hodnoty user_id vždy patria do rovnakého bucketu.
- Rôzne hodnoty môžu skončiť v rovnakom buckete: $h(99) = h(42) = 0$.

Hash join — build + probe

1. Build: vytvor hašovaciu tabuľku v RAM. Prejdi S a každý riadok s vlož do bucketu $h(S.\text{user_id})$.
2. Probe: postupne čítaj riadky R. Pre každý r vypočítaj $h(R.\text{user_id})$, prejdi daný bucket a vypíš všetky dvojice s rovnakou hodnotou user_id.

Väčšiu reláciu R prechádzame iba raz.

udalosť	prehľadáme bucket	výsledok
$(e_1, 17)$	2	$(e_1,$ Eva)
$(e_2, 42)$	0	$(e_2,$ Adam)
$(e_3, 17)$	2	$(e_3,$ Eva)
$(e_4, 99)$	0	žiadna zhoda

Hašujeme hodnoty user_id oboch vstupov, ale ukladáme iba S. Nepotrebujeme utriedenie ani vopred vytvorený index.

Hash join — query plan

Výpis z `EXPLAIN ANALYZE` (PostgreSQL 16):

```
Hash Join (actual time=0.450..31.079 rows=200000 loops=1)
  Hash Cond: (r.user_id = s.user_id)
  ->  Seq Scan on udalosti r (actual time=0.005..8.308 rows=200000 loops=1)
  ->  Hash (actual time=0.442..0.444 rows=4000 loops=1)
        Buckets: 4096  Batches: 1  Memory Usage: 216kB
        ->  Seq Scan on pouzivatelia s (actual time=0.002..0.194 rows=4000 loops=1)
Execution Time: 35.806 ms
```

- zelená: build — menšia relácia S sa celá zmestí do jednej dávky v pamäti;
- červená: probe — 200 000 riadkov R sa prejde raz;
- modrá: podmienka rovnosti na `user_id`.
- Strany vyberá plánovač podľa odhadnutej veľkosti; vstupom môže byť aj výsledok filtrovania či inej operácie.

Hash join — veľkosť výsledku

- Výsledok môže byť veľký: jedna hodnota `user_id` s 1000 udalosťami a 100 záznamami vytvorí 100 000 riadkov, preto pozor na duplicity.
- Avšak výsledok nezávisí od spôsobu výpočtu, preto si pri optimalizácii nevšímame napr. čas zápisu výsledku na disk.
- Ak sa jedna z tabuliek zmestí do RAM, oba vstupy prejdeme raz, čiže počet prečítaných diskových blokov je minimálny možný.

Hash join — pipelining

- Zhody možno odosielať na výstup priebežne, nečaká sa na celý výsledok.
- V EXPLAIN pláne: `Hash Join` vráti prvý riadok hneď po dokončení uzla `Hash`.
- Vstup R (probe) môže byť priebežný: každý riadok sa hneď porovná s hašovacou tabuľkou.
- Vstup S (build) je blokujúci: celý sa musí prečítať do hašovacej tabuľky, až potom vzniká prvý výstup.

Hash join — dávky

- Ak sa nezmestí do RAM ani jedna zo vstupných relácií, rozdelíme obe rovnakou hašovacou funkciou na dávky (časti vstupu, batches) uložené na disku.
- Dávky vyberáme z tej istej hashovacej funkcie, ale len niekoľkými bitmi hashu; vnútorné buckety v pamäti používajú iné bity. Počet dávok je zvyčajne malý (na rozdiel od počtu bucketov), ak by ich bolo priveľa, bloky na disku budú príliš prázdne. Ak by dávky použili tie isté bity ako buckety, všetky riadky v jednej dávke by mali rovnaké tieto bity a hašovacia tabuľka by využívala len časť bucketov.
- Rovnaká hodnota user_id skončí na oboch stranách v rovnakej dávke, takže žiadnu zhodu nevynecháme.
- Pre každú dvojicu zodpovedajúcich si dávok aplikujeme predošlý algoritmus.
- Cena navyše: zápis dávok na disk a ich opätovné čítanie,

Hash join — query plan with batches

Výpis z `EXPLAIN ANALYZE` (PostgreSQL 16),
ten istý join s rovnakými dátami, ale `work_mem` = 64 kB:

```
Hash Join (actual time=0.815..48.337 rows=200000 loops=1)
  Hash Cond: (r.user_id = s.user_id)
  ->  Seq Scan on udalosti r (actual time=0.006..8.422 rows=200000 loops=1)
  ->  Hash (actual time=0.681..0.681 rows=4000 loops=1)
        Buckets: 2048  Batches: 8  Memory Usage: 37kB
        ->  Seq Scan on pouzivatelia s (actual time=0.004..0.227 rows=4000 loops=1)
Execution Time: 53.129 ms
```

- Batches: 8 — tabuľka `S` sa rozdelila na 8 dávok; join trvá 53 ms oproti 36 ms bez dávok.
- V RAM je naraz len jedna dávka `S` ako hašovacia tabuľka, zodpovedajúca dávka `R` sa číta postupne.

Zdroj: `https://www.postgresql.eu/events/pgconfeu2019/sessions/session/2669/slides/226/2019_HashJoin_In_PostgreSQL_Milan.pdf`

Hash join — známe limity

- Skew: veľa riadkov s rovnakou hodnotou zostane v jednej dávke, lebo žiadne hašovanie nerozdelí rovnakú hodnotu.
- PostgreSQL postup pre skew: najčastejšie hodnoty sa spracujú vopred osobitne, nie v dávkach.
- Keď vytvárame dávky, každá dávka má vlastný výstupný buffer v pamäti. Pri veľkom počte dávok tieto buffery zaberú viac, než dovoľuje `work_mem`.

Fyzické operátory — join v SQLite

- SQLite nemá hash join. Pri joine dočasne vytvorí B-strom (index) nad join stĺpcom a v ňom vyhľadáva. V ideálnom prípade indexy pre obe joinované tabuľky už existujú.
- Ide o nested loop s vyhľadávaním, nie o merge join; každé vyhľadanie je logaritmické. (Pre málo dát nevadí.)
- Dôvod: SQLite beží aj na malých zariadeniach a B-strom beztak potrebuje, takže hashovacia tabuľka by celú knižnicu (jeden spustiteľný súbor, okolo 1 MB) zbytočne zväčšila.

Zdroj: `https://www.sqlite.org/optoverview.html#automatic_indexes`

Fyzické operátory — nested loop join

Počítame nested loop join pre R a S, pričom vo vonkajšom cykle prechádzame cez S a vo vnútornom cez R. Pamäť: M blokov.

Rezervujeme 1 blok pre výstup. Načítame $M-2$ blokov S a 1 blok rezervujeme na čítanie R. Postupne čítame všetky bloky z R, vyhodnocujeme joinovaciu podmienku a výsledky zapisujeme na výstup (každý blok R prečítame raz bez ohľadu na počet blokov alokovaných pre R, preto nemá význam alokovať viac ako 1 blok). Ak má S ďalšie bloky, načítame $M-2$ blokov S a zopakujeme čítanie celej R po 1 bloku. Počet opakovaných čítaní R je $\lceil b(S)/(M-2) \rceil$.

Celková cena je $b(S) + \lceil b(S)/(M-2) \rceil \cdot b(R)$. Nezahŕňa zapisovanie výstupu, ale jeho veľkosť nezávisí od spôsobu výpočtu joinu. Oplatí sa vo vonkajšom cykle iterovať cez menšiu z R, S.

Fyzické operátory — nested loop join

Úloha. Počítame dotaz

```
SELECT A.x FROM A, B WHERE A.x = B.x
```

Relácia A je na disku uložená v 100 blokoch, relácia B v 50 blokoch. K dispozícii v RAM je 20 blokov (bloky na disku a v RAM sú rovnako veľké).
(a) Zvoľte poradie vonkajšej a vnútornej relácie, popíšte výpočet metódou nested loop join a zistite počet potrebných I/O operácií.
(b) Zdôvodnite, prečo je pri nested loop join optimálne použiť pre vnútornú reláciu len 1 blok a zvyšok RAM využiť na chunk vonkajšej relácie.

Literatúra

Disky

- https://cs186berkeley.net/notes/note3/
- https://www.db-book.com/slides-dir/PDF-dir/ch12.pdf
- https://www.db-book.com/slides-dir/PDF-dir/ch13.pdf

Spracovanie dotazov a operátory

- https://cs186berkeley.net/notes/note8/
- https://cs186berkeley.net/notes/note9/
- https://www.db-book.com/slides-dir/PDF-dir/ch15.pdf

Na zamyslenie

- Aké fyzické operátory zodpovedajú množinovému rozdielu?
- Popíšte, ako možno nested loop join 4 tabuliek vypočítať bez ukladania medzivýsledkov.
- Rátame nested loop join pre relácie, ktoré zaberajú 1000 a 20 blokov. Najmenej koľko blokov v RAM potrebuje DBMS alokovať na výpočet, aby bol počet diskových čítaní minimálny?